SAMS Nepal

School Asset Management System
Architecture & Foundation Plan

Enterprise Architecture Document

A centralized Government School Asset Management System for Nepal.
Portfolio-quality project inspired by real enterprise asset management software.

Document Version: 1.0
Date: August 18, 2026
Status: Pending Approval

Table of Contents

1. Reference Project Analysis (Updated/)

Current Structure

```
Updated/
├── frontend/
│   ├── index.html          # ~27,000 lines — single-file SPA
│   └── Dockerfile          # nginx static serve
├── backend/
│   ├── server.js           # Express entry, route mounting
│   ├── db.js               # Hardcoded pg Pool
│   ├── routes/
│   │   ├── tmi.js          # ~900 lines — full CRUD + import + audit
│   │   ├── testInfra.js
│   │   ├── officeInfra.js
│   │   └── calibration.js
│   └── Dockerfile
├── database/
│   └── init.sql            # 3 duplicate asset tables + history/audit/service
└── docker-compose.yml      # postgres + backend + frontend
```

Reference Characteristics

- **Frontend:** Single monolithic HTML file (~27K lines) with inline CSS and JavaScript — no framework, no component modularity.
- **Backend:** Express.js with route-per-domain pattern; duplicated CRUD logic across TMI, Test Infra, and Office Infra routes.
- **Database:** Three separate asset tables with SERIAL primary keys; no users/auth tables; JSON payload columns for flexibility.
- **Authentication:** Client-side only with hardcoded user array; no JWT, no backend security.
- **Layout:** Collapsible sidebar, top header, page switching via showPage/showSubPage functions, login screen overlay.
- **Features:** Asset CRUD, search, bulk Excel import, audit records, service records, history, modals, wizards, KPI stat cards, filtered tables, reporting.
- **Docker:** Three-service stack — PostgreSQL, backend, nginx frontend.

2. **Conceptual Patterns to Reuse**

Pattern	Reference Implementation	SAMS Application
Layout shell	Collapsible sidebar + top header + main content	Same shell, rebuilt in Vue components
Page routing	showPage() / showSubPage() toggles .page.active	Vue Router with nested routes
Navigation groups	Expandable sidebar sections	Role-filtered nav groups
Login gate	Full-screen login overlay before app shell	Dedicated /login route + auth guard
Dashboard KPIs	.stat-card grid with color-coded metrics	Pinia-driven KPI cards + Chart.js
Asset tables	Search, column filters, pagination, modal CRUD	Reusable DataTable + AssetFormModal
Add asset flow	Method picker → wizard OR Excel import	Modal form first; import in Phase 2
Audit trail	Separate history/audit/service record tables	Unified asset_history + future maintenance module
Docker dev stack	postgres → backend → nginx frontend	Same topology, env-driven config

3. **Improvements Over Reference**

Area	Reference Problem	SAMS Improvement
Frontend	27K-line monolith, no framework, unmaintainable	Vue 3 + Vite modular architecture
Auth	Client-side hardcoded users, no API security	JWT + bcrypt + server-side RBAC
Database	3 duplicate asset tables, SERIAL IDs, no FK relations	Single normalized assets table, UUID PKs
Domain model	Industrial lab assets (TMI/Test Infra)	Government school hierarchy (State → Municipality → School)
Multi-tenancy	None	Role-scoped data filtering by municipality/school
Config	Hardcoded DB credentials in db.js	Environment variables (.env, Neon, Render)
API design	Fat route files, no validation layer	Controllers → Services → Repositories
Styling	5,000+ lines inline CSS, theme variants	Tailwind CSS with design tokens
Type safety	None	JSDoc or optional TypeScript in backend
Testing	None	Testable service layer from day one

4. Proposed Project Structure

```
Asset-Management/
├── frontend/                          # Vue 3 SPA → Cloudflare Pages
│   ├── public/
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   │   ├── layout/                # AppLayout, AppSidebar, AppHeader
│   │   │   ├── ui/                    # BaseButton, BaseCard, BaseModal, etc.
│   │   │   ├── dashboard/             # KpiCard, Charts
│   │   │   ├── assets/                # AssetTable, AssetFilters, AssetFormModal
│   │   │   └── schools/               # MunicipalityList, SchoolCard
│   │   ├── composables/               # useAuth, usePermissions, useApi
│   │   ├── layouts/                   # AuthLayout, DashboardLayout
│   │   ├── router/                    # Routes + guards + meta.roles
│   │   ├── stores/                    # auth, ui, dashboard (Pinia)
│   │   ├── services/                  # API service modules
│   │   ├── views/                     # Page views by module
│   │   ├── App.vue, main.js, style.css
│   ├── index.html, vite.config.js, tailwind.config.js, package.json
│
├── backend/                           # Express API → Render
│   ├── src/
│   │   ├── config/                    # Env loader, database pool
│   │   ├── middleware/                # auth, authorize, validate, errorHandler
│   │   ├── routes/                    # Route definitions
│   │   ├── controllers/               # Request handlers
│   │   ├── services/                  # Business logic
│   │   ├── repositories/              # SQL queries
│   │   ├── utils/                     # jwt, password, scope
│   │   ├── constants/                 # roles, permissions
│   │   └── app.js
│   ├── server.js, Dockerfile, package.json
│
├── database/
│   ├── schema/                        # 001–009 migration SQL files
│   ├── seeds/                         # Roles, municipalities, schools, categories
│   └── migrations/                    # Future: node-pg-migrate
│
├── docker-compose.yml
├── .gitignore
└── README.md
```

5. Database Schema

Core Tables

Table	Purpose	Key Fields
roles	User role definitions	id (UUID), name, permissions (JSONB)
municipalities	Local government units	id, name, province, district
schools	Government schools	id, municipality_id (FK), name, school_code
users	System users	id, role_id, municipality_id, school_id, email, password_hash
asset_categories	Asset classification	id, name, department
asset_statuses	Asset lifecycle states	id, name, color_code, sort_order
assets	School assets	id, asset_tag, name, category_id, school_id, status_id, purchase_cost, etc.

Entity Relationships

- roles → users (one-to-many)
- municipalities → schools (one-to-many)
- municipalities → users (one-to-many, for municipal officers)
- schools → users (one-to-many, for school admins)
- schools → assets (one-to-many)
- asset_categories → assets (one-to-many)
- asset_statuses → assets (one-to-many)
- users → assets (created_by reference)

Asset Fields

Field	Type	Description
asset_tag	VARCHAR(50) UNIQUE	Human-readable asset identifier
name	VARCHAR(300)	Asset name
category_id	UUID FK	Asset category
school_id	UUID FK	Owning school
status_id	UUID FK	Current status
department	VARCHAR(100)	Department within school
location	VARCHAR(200)	Physical location
purchase_date	DATE	Date of purchase
purchase_cost	NUMERIC(12,2)	Purchase cost in NPR
warranty_expiry	DATE	Warranty expiration
vendor	VARCHAR(200)	Supplier/vendor name
deleted_at	TIMESTAMPTZ	Soft delete timestamp

Seed Data Summary

Seed File	Records
001_roles.sql	3 roles: state_admin, municipal_officer, school_admin
002_municipalities.sql	3 municipalities (Butwal, Siddharthanagar, Lumbini Sanskritik)
003_schools.sql	27 schools across 3 municipalities
004_asset_categories.sql	8 parent categories + 28 sub-items
005_asset_statuses.sql	Active, Damaged, Under Maintenance, Disposed, Lost
006_demo_users.sql	1 demo user per role for development

Municipalities & Schools (Seed Data)

1. Butwal Sub-Metropolitan City (10 schools)

- Kalika Manavgyan Secondary School
- Kanti Secondary School
- Naharpur Secondary School
- Nayagaun Secondary School

- Nawa Ratna Secondary School
- Nabin Audhyogic Kadar Bahadur Rita Secondary School
- Jitgadi Basic School
- Amar Secondary School
- Sitaram Basic School
- Ujyaalo Basic School

2. Siddharthanagar Municipality / Bhairahawa (7 schools)

- Bhairahawa Model Secondary School
- Rupandehi Secondary School
- Paklihawa Secondary School
- Siddhartha Secondary School
- Gautam Buddha Secondary School
- Kanya Secondary School
- Narainapur Basic School

3. Lumbini Sanskritik Municipality (10 schools)

- Tenuhawa Community Secondary School
- Khudabagar Secondary School
- Shree Masina Baba Narendra Puri Moglaha Secondary School
- Balrampur Secondary School
- Karmahawa Secondary School
- Aama Secondary School
- Nepal Rastriya Basic School
- Bhagwanpur Secondary School
- Sonbarsha Basic School
- Mahilwar Secondary School

Asset Categories

Department	Categories
Classroom Assets	Benches, Desks, Chairs, Whiteboards, Smart Boards, Projectors
Computer Lab Assets	Desktop Computers, Laptops, Printers, UPS, Routers
Library Assets	Books, Bookshelves, Computers
Science Lab Assets	Microscopes, Lab Equipment
Sports Assets	Footballs, Cricket Kits, Volleyball Equipment
Office Assets	Computers, Printers, Tables, Chairs
Infrastructure Assets	CCTV Cameras, Water Coolers, Solar Panels, Generators

6. Backend API Structure

Base URL

```
Production:  https://sams-api.onrender.com/api/v1
Development: http://localhost:5000/api/v1
```

Endpoint Map

Method	Endpoint	Access
POST	/auth/login	Public
POST	/auth/logout	Authenticated
GET	/auth/me	Authenticated
GET	/dashboard/kpis	All roles (scoped)
GET	/dashboard/charts/municipality	State Admin, Municipal Officer
GET	/dashboard/charts/category	All roles (scoped)
GET	/dashboard/charts/status	All roles (scoped)
GET/POST/PUT	/municipalities	State Admin
GET/POST/PUT	/schools	Scoped by role
GET/POST/PUT/DELETE	/assets	Scoped by role
GET/POST/PUT	/categories	Read: all; Write: State Admin
GET/POST/PUT/DELETE	/users	State Admin
GET	/reports/*	Scoped by role

Request/Response Conventions

```
// Success envelope
{ "success": true, "data": { ... }, "meta": { "page": 1, "limit": 20, "total": 142 } }

// Error envelope
{ "success": false, "error": { "code": "FORBIDDEN", "message": "..." } }

// Asset list query params
GET /assets?page=1&limit=20&search=projector&category_id=uuid&status_id=uuid
```

Backend Layer Flow

```
Request
  → auth.middleware        (verify JWT)
  → authorize.middleware   (check role + permission)
  → validate.middleware    (express-validator)
  → controller             (parse req, call service, send res)
  → service                (business logic, scope filtering)
  → repository             (parameterized SQL queries)
  → PostgreSQL
```

7. Frontend Routing Structure

Route Tree

```
/                          → redirect to /dashboard or /login
/login                     → LoginView                    [guest only]
/dashboard                 → DashboardView                [all roles]
/assets                    → AssetListView                [all roles, scoped]
/assets/:id                → AssetDetailView              [all roles, scoped]
/schools                   → MunicipalityListView         [state_admin, municipal_officer]
/schools/municipality/:id  → SchoolListView               [state_admin, municipal_officer]
/schools/:id               → SchoolDetailView             [all roles, scoped]
/reports                   → ReportsView                  [all roles, scoped]
/users                     → UserManagementView           [state_admin]
/categories                → CategoryManagementView       [state_admin]
```

Navigation by Role

Nav Item	State Admin	Municipal Officer	School Admin
Dashboard	✓	✓	✓
Assets	✓ (all)	✓ (municipality)	✓ (own school)
Schools	✓	✓ (own municipality)	✓ (own school)
Reports	✓	✓	✓
Users	✓	—	—
Categories	✓	—	—

8. Role-Based Access Architecture

User Roles & Permissions

1. State Administrator

- Manage all districts and municipalities
- Manage all schools
- View all assets
- View analytics and generate reports
- Manage users and asset categories

2. Municipal Officer

- View schools inside assigned municipality

- Audit assets within municipality
- Approve maintenance requests (Phase 2)
- Generate municipality reports

3. School Administrator

- Manage school assets (own school only)
- Add, edit assets
- Create maintenance requests (Phase 2)
- View school reports

Permission Constants

```
PERMISSIONS:
  municipalities:read, municipalities:write
  schools:read, schools:write
  assets:read, assets:write, assets:delete
  categories:write
  users:read, users:write
  reports:read, dashboard:read
  maintenance:approve (Phase 2)
```

Data Scoping Strategy

Role	SQL Scope Clause
state_admin	No filter — access all data
municipal_officer	s.municipality_id = user.municipality_id
school_admin	a.school_id = user.school_id

JWT Payload

```
{
  "sub": "user-uuid",
  "email": "admin@sams.gov.np",
  "role": "state_admin",
  "municipality_id": null,
  "school_id": null,
  "iat": 1700000000,
  "exp": 1700086400
}
```

9. UI Design System

Design Principles

- Modern, professional, clean government-enterprise grade appearance
- Left collapsible sidebar + top navigation header + main content area
- Responsive design for desktop, tablet, and mobile
- Card-based dashboard with proper spacing and consistent typography
- Enterprise tables and professional forms with subtle animations only

Color Palette

Token	Value	Usage
Primary 500	#2563EB	Brand, buttons, links
Primary 700	#1D4ED8	Hover states
Slate 50	#F8FAFC	Page background
Slate 900	#0F172A	Sidebar background
Success	#16A34A	Active status
Warning	#D97706	Maintenance status
Danger	#DC2626	Damaged status

Layout Dimensions

Element	Size
Sidebar (expanded)	240px
Sidebar (collapsed)	64px
Header height	56px
Content max-width	1440px
Card border-radius	8px
Font family	Inter (Google Fonts)

Visual Differentiation from Reference

Aspect	Reference (Updated/)	SAMS Nepal
Sidebar	White background	Dark slate-900 sidebar
Cards	Flat with colored stat dots	White cards with shadow + accent border
Tables	Dense inline filter bars	Spacious rows, sticky header, external filters
Typography	IBM Plex Sans	Inter
Login	Centered card on gradient	Split-panel: branding left, form right
Icons	Inline SVG strings	Heroicons via @heroicons/vue

Dashboard KPIs

- Total Assets
- Active Assets
- Damaged Assets
- Under Maintenance
- Total Schools
- Total Asset Value

Dashboard Charts

- Assets by Municipality (bar chart)
- Assets by Category (donut chart)
- Asset Status Distribution (pie chart)

10. Deployment Architecture

Production Stack

Component	Platform	Notes
Frontend (Vue 3 SPA)	Cloudflare Pages	Static build from Vite
Backend (Express API)	Render	Web service with health check
Database (PostgreSQL)	Neon	Serverless Postgres with SSL
Local Development	Docker Compose	postgres:16 + backend + Vite dev

Environment Variables

```
Frontend (.env):
  VITE_API_BASE_URL=https://sams-api.onrender.com/api/v1

Backend (.env):
  NODE_ENV=production
  PORT=5000
  DATABASE_URL=postgresql://user:pass@ep-xxx.neon.tech/sams?sslmode=require
  JWT_SECRET=<random-256-bit>
  JWT_EXPIRES_IN=24h
  CORS_ORIGIN=https://sams-nepal.pages.dev
```

11. Implementation Phases

Phase	Scope	Deliverables
Phase 1 — Foundation	Project scaffolding	Folder structure, Docker, DB schema + seeds, auth API, login page, app shell
Phase 2 — Core Modules	Dashboard + Assets + Schools	KPIs, charts, asset CRUD, school listing, scoped queries
Phase 3 — Admin	Users + Categories + Reports	User management, category CRUD, CSV export
Phase 4 — Polish	Production readiness	Error handling, loading states, responsive QA, deployment configs

12. Decisions Needing Approval

> The following decisions should be confirmed before implementation begins.

1. **Sidebar style** — Dark sidebar (slate-900) vs. light sidebar (white, like reference)?
2. **Asset tag format** — Auto-generated (SAMS-BTW-2026-0001) or manual entry?
3. **Maintenance module** — Include maintenance_requests table in Phase 1 schema (empty), or defer to Phase 2?
4. **Demo users** — Seed with bcrypt passwords for all 3 roles?
5. **Chart library** — Chart.js (lightweight) or ApexCharts (richer)?
6. **Monorepo root** — Create frontend/, backend/, database/ at project root (alongside Updated/), or replace Updated/?

13. **Tech Stack Summary**

Layer	Technology
Frontend Framework	Vue 3
Build Tool	Vite
Routing	Vue Router
State Management	Pinia
CSS Framework	Tailwind CSS
Charts	Chart.js
Icons	@heroicons/vue
Backend Runtime	Node.js
Backend Framework	Express.js
Database	PostgreSQL (UUID primary keys)
Authentication	JWT + bcrypt
Containerization	Docker / Docker Compose
Frontend Deployment	Cloudflare Pages
Backend Deployment	Render
Database Hosting	Neon PostgreSQL